

Solution Write-Up

Project Title

Podcast Q&A; Bot using Retrieval-Augmented Generation (RAG)

Problem Statement

Podcasts and long-form YouTube videos contain valuable information, but extracting specific information from lengthy content is often difficult and time-consuming. Users typically need to manually search through long recordings to find answers to particular questions. The objective of this project was to develop an AI-powered system capable of understanding podcast content and allowing users to ask natural language questions about a YouTube video. The system should provide accurate answers along with supporting transcript evidence and timestamps from the original source.

Proposed Solution

To address this problem, a Retrieval-Augmented Generation (RAG) architecture was implemented. The solution combines YouTube audio extraction, speech-to-text transcription, semantic retrieval, vector databases, and large language models to enable intelligent question answering over podcast content. Instead of relying solely on a language model's knowledge, the system retrieves relevant transcript segments and uses those segments as context for answer generation. This approach improves answer accuracy and reduces hallucinations.

System Architecture

Workflow:

- Download audio from YouTube videos using yt-dlp
- Convert audio into WAV format using FFmpeg
- Split long audio into manageable chunks
- Generate timestamped transcripts using Faster Whisper

- Create embeddings using Sentence Transformers
 - Store embeddings in FAISS vector database
 - Retrieve relevant transcript chunks using semantic search
 - Generate answers using Google Gemini 2.5 Flash
 - Display timestamps and transcript evidence in Streamlit
-

Implementation Details

1. Audio Extraction

The user provides a YouTube URL. Audio is downloaded using yt-dlp and converted into WAV format using FFmpeg.

2. Speech-to-Text Transcription

Audio is transcribed using Faster Whisper. To avoid memory issues with long podcasts, audio is automatically divided into 10-minute chunks. Each transcript segment contains:

- Text
- Start Timestamp
- End Timestamp

3. Embedding Generation

Each transcript segment is converted into dense vector representations using:

- all-MiniLM-L6-v2

4. Vector Database Creation

Generated embeddings are stored inside a FAISS vector index. FAISS enables efficient semantic similarity search over transcript segments.

5. Semantic Retrieval

When a user asks a question:

1. The query is converted into an embedding.
2. FAISS performs similarity search.

3. Top relevant transcript chunks are retrieved.

6. Answer Generation

Retrieved transcript chunks are passed to Google Gemini 2.5 Flash through a LangChain Retrieval-Augmented Generation pipeline. Gemini generates answers using only the retrieved transcript context.

7. Caching Mechanism

To reduce processing time, the system caches:

- Downloaded audio files
- Generated transcripts
- FAISS indexes
- Transcript stores

8. Answer Verification

The system displays:

- Generated answer
 - Supporting transcript evidence
 - Timestamp of the best matching segment
 - Direct YouTube navigation link This allows users to verify answers directly against source content.
-

Key Features

- AI-powered Podcast Question Answering
 - Retrieval-Augmented Generation (RAG)
 - Semantic Search using FAISS
 - Faster Whisper Transcription
 - Timestamp-Based Navigation
 - Transcript Evidence Display
 - Audio Caching
 - Transcript Caching
 - FAISS Index Caching
 - Streamlit Chat Interface
-

Challenges Faced

Retrieval Quality

Ensuring that retrieved transcript segments were relevant to user questions was critical. Semantic embeddings and optimized similarity search were used to improve retrieval quality.

Long Video Processing

Podcast episodes can be lengthy and computationally expensive to process. Audio chunking and caching mechanisms were introduced to reduce repeated computation.

Answer Grounding

Generated responses needed to remain faithful to source content. Retrieved transcript chunks are supplied before answer generation to ensure contextual accuracy.

User Verification

Supporting transcript evidence and timestamps are displayed so users can verify generated answers.

Results

The developed system successfully enables users to:

- Process YouTube videos automatically
- Generate timestamped transcripts
- Ask natural language questions
- Receive context-aware answers
- View transcript evidence
- Navigate directly to relevant moments in the source video

The solution demonstrates the practical application of Retrieval-Augmented Generation for long-form audio and video content.

Technologies Used

Programming Language

- Python

Frontend

- Streamlit

Audio Processing

- yt-dlp
- FFmpeg

Speech Recognition

- Faster Whisper

Embeddings

- Sentence Transformers
- all-MiniLM-L6-v2

Vector Database

- FAISS

Large Language Model

- Google Gemini 2.5 Flash

Frameworks

- LangChain

Environment Management

- python-dotenv
 - Virtual Environment
-

Future Enhancements

- Multi-video knowledge base
 - Hybrid retrieval
 - Speaker diarization
 - Multi-language support
 - Conversation memory
 - Podcast summarization
 - Analytics dashboard
 - Cloud deployment
-

Conclusion

This project demonstrates how Retrieval-Augmented Generation can transform long-form YouTube and podcast content into an interactive and searchable knowledge source. By combining Faster Whisper transcription, semantic retrieval using FAISS, Sentence Transformers, and Gemini-based answer generation, the system provides accurate, explainable, and user-friendly access to information contained within lengthy videos.
